

AI Basics for Developers

Modul 1 · Key Fact Cards

2 Stunden · 4C-orientiert · Theorie → Übung → Reflexion

Lernziele

Nach dem Modul kann der Teilnehmer ...

- erklären, was ein LLM grundsätzlich tut und warum Output nicht automatisch wahr ist,
- die Rolle von **Tokens**, **Context Window** und **Kontext** beschreiben,
- typische Grenzen wie **Halluzinationen** und Scheinsicherheit erkennen,
- **Prompting**, **Context Engineering**, **RAG**, **Tool Calling** und **Agenten** unterscheiden,
- einschätzen, wann ein LLM geeignet ist und wann klassische Softwaremechanismen besser passen,
- einfache Aufgaben so strukturieren, dass ein LLM verlässlicher damit arbeiten kann.

Didaktischer Rhythmus

Training from the Back of the Room · 4C

Connections → Vorwissen und Erfahrung aktivieren

Concepts → kurzer Theorieimpuls

Concrete Practice → Konzept sofort anwenden

Conclusions → Erkenntnisse sichern und transferieren

Designregel

■ Kein Theorieblock länger als ca. 10–12 Minuten ohne Aktivität der Teilnehmer.

1 · AI im Entwickleralltag: Wo stehen wir?

Timing: 10 Minuten

4C: Connections

Einstieg

■ Wo nutzt ihr heute bereits AI in eurer Arbeit als Entwickler und wo funktioniert sie überraschend gut oder überraschend schlecht?

Concepts

Noch kein eigentlicher Theorieblock. Sichtbar machen:

- Wo hilft AI heute schon?
- Wo entstehen Frust oder Unsicherheit?
- Welche Erwartungen haben die Teilnehmer?
- Welche wiederkehrenden Muster sehen wir bereits?

Concrete Practice

Pair-Austausch, 2–3 Minuten

1. Wo nutzt du heute AI?
2. Was funktioniert gut?
3. Wo hattest du schlechte oder überraschende Ergebnisse?
4. Was möchtest du besser verstehen?

Danach kurze Sammlung im Plenum: „Hilft“ / „Nervt“.

Übergang

2 · Plausibel ist nicht gleich richtig

Timing: ca. 20–22 Minuten

4C: Concepts → Concrete Practice

Einstieg

Der Himmel ist ...

oder

```
if (user == null)
{
    ...
}
```

Was wäre eine plausible Fortsetzung? Gibt es nur eine richtige?

Concepts

- Sprache und Code werden in **Tokens** zerlegt.
- Ein LLM erzeugt schrittweise wahrscheinliche Fortsetzungen.
- Es arbeitet mit statistisch gelernten Mustern.
- **Training ≠ Inference.**
- Das Modell sucht beim Antworten nicht automatisch Fakten nach.
- Output ist **probabilistisch**, nicht deterministisch.

Ein LLM ist eine sehr leistungsfähige Generierungsmaschine, keine Wahrheitsmaschine.

2 · Concrete Practice: Be the LLM

Timing: ca. 10 Minuten

Übung

Pairs oder kleine Gruppen erhalten mehrere Fragmente:

Paris ist die Hauptstadt von ...

```
public bool IsValid(Order order)
{
```

```
SELECT * FROM users WHERE
```

Auftrag

1. Notiert mehrere plausible Fortsetzungen.
2. Entscheidet, welche am wahrscheinlichsten wirkt.
3. Diskutiert: Bedeutet **wahrscheinlich** gleichzeitig **korrekt**?
4. Optional: Schickt denselben Prompt an ein LLM und vergleicht.

Übergang

■ Wenn ein Modell auf Plausibilität optimiert und nicht auf Wahrheit, kann es sehr überzeugend klingen und trotzdem falsch liegen.

3 · Confidently Wrong: Halluzinationen

Timing: ca. 20 Minuten

4C: Concepts → Concrete Practice

Einstieg

Explain the `AsyncQuantumCache` API introduced in .NET 10 and show me how to use `EnableQuantumInvalidation()`.

Beobachten: Hinterfragt das Modell die Prämisse oder baut es eine plausible Geschichte?

Concepts

- Halluzinationen ergeben sich aus der Funktionsweise, nicht nur aus „schlechtem Prompting“.
- Fehlende Information führt nicht automatisch zu „Ich weiß es nicht“.
- Modelle können falsche Prämissen übernehmen.
- Sprachliche Sicherheit ist **kein Beweis für faktische Sicherheit**.
- Kritisch sind insbesondere unbekannte APIs, Versionen, Libraries und nicht vorhandener Codekontext.
- **Verification** ist Teil des Engineering-Prozesses.

3 · Concrete Practice: Make it hallucinate

Timing: ca. 10 Minuten

Übung

Jedes Pair versucht bewusst, sein LLM aufs Glatteis zu führen.

1. Erfindet eine glaubwürdig klingende Library, API oder Behauptung.
2. Fragt das Modell danach.
3. Beobachtet:
 - Hinterfragt es die Information?
 - Erfindet es Details?
 - Wie sicher formuliert es?
4. Verändert danach den Prompt, z. B.:
 - „If you are uncertain, state that explicitly.“
 - „Do not assume this API exists.“
 - „Separate verified facts from assumptions.“

Debrief

■ Was konnten wir verbessern und was konnten wir trotzdem nicht garantieren?

Übergang

■ Wir können das Modell nicht einfach darum bitten, korrekt zu sein. Eine stärkere Stellschraube ist der Kontext, den wir ihm geben.

4 · Prompting → Context Engineering

Timing: ca. 25–28 Minuten

4C: Concepts → Concrete Practice

Einstieg

A: „Implementiere eine Funktion zur Validierung einer Bestellung.“

B: Dieselbe Aufgabe plus Datenstruktur, Geschäftsregeln, Constraints, Edge Cases, Architektur und Akzeptanzkriterien.

■ Könnte ein schwächeres Modell mit gutem Kontext besser abschneiden als ein stärkeres Modell ohne Kontext?

Concepts

Prompting: Wie formuliere ich meine Anweisung?

Context Engineering: Welche relevanten Informationen hat das Modell überhaupt zur Verfügung?

Relevanter Kontext kann sein:

- Instructions
- Code und Dateien
- Requirements und Constraints
- Beispiele und Edge Cases
- Architektur
- Akzeptanzkriterien
- vorherige Conversation

■ Mehr Kontext ist nicht automatisch besser. Relevanter Kontext ist besser.

4 · Concrete Practice: Same task, better context

Timing: ca. 15 Minuten

Runde 1

■ Implementiere eine Funktion zur Validierung einer Bestellung.

Prompt direkt abschicken.

Runde 2 · zusätzlicher Kontext

- `Order` enthält Positionen, Kundennummer und Gesamtbetrag.
- Mindestens eine Position ist erforderlich.
- Menge muss > 0 sein.
- Negative Preise sind nicht zulässig.
- Bestellungen > €10.000 benötigen zusätzliche Freigabe.
- Die Funktion darf keine Exception werfen.
- Validierungsfehler sollen strukturiert zurückgegeben werden.

Auswertung

- Welche Annahmen verschwanden?
- Welche Information hatte den größten Einfluss?
- Welche Entscheidungen hat das Modell trotzdem selbst getroffen?

Übergang

■ Bisher versuchen wir, das Modell besser arbeiten zu lassen. Aber die nächste Frage lautet: Muss das Modell diese Aufgabe überhaupt selbst erledigen?

Pause

ca. 10 Minuten

■ Bevor wir das Modell mit Tools ausstatten, statten wir uns kurz mit Kaffee aus.

5 · Nicht alles gehört ins LLM

Timing: ca. 22 Minuten
4C: Concepts → Concrete Practice

Einstieg

Ein Mitarbeiter fragt einen Chatbot: „Wie viele Urlaubstage habe ich noch?“

Was sollte das LLM tun?

Concepts

Baustein	Geeignet für
LLM	Interpretieren, Klassifizieren, Zusammenfassen, Generieren
Retrieval / RAG	relevante Informationen aus Wissensquellen holen
Tools	Daten abfragen und Aktionen durchführen
klassischer Code	deterministische Regeln, Berechnungen, Validierung
Human-in-the-Loop	Verantwortung, Unsicherheit, Freigaben

Agenten: Ein LLM kann innerhalb eines kontrollierten Prozesses entscheiden, welche Tools oder Aktionen als Nächstes verwendet werden.

5 · Concrete Practice: Design the AI Boundary

Timing: ca. 12 Minuten

Szenarien

- Mitarbeiter fragt nach Resturlaub.
- Entwickler möchte eine unbekannte Codebasis verstehen.
- Rechnungssumme berechnen.
- Supportticket kategorisieren.
- Produktionsdeployment auslösen.
- 300 technische Dokumente nach Anforderungen durchsuchen.

Auftrag

Entscheidet für jedes Szenario:

LLM · Retrieval · Tool · klassischer Code · Mensch

Es geht nicht um die perfekte Architektur, sondern um die **Begründung der Grenze**.

Debrief

2–3 kontroverse Entscheidungen kurz im Plenum diskutieren.

Übergang

■ Wir haben jetzt einen Werkzeugkasten. Welche Regeln nehmen wir daraus für unseren eigenen Umgang mit AI mit?

6 · Was nehme ich morgen mit?

Timing: 8–10 Minuten

4C: Conclusions

Einstieg

■ Bevor ich euch sage, was ihr euch merken sollt: Was ist für euch heute die wichtigste Erkenntnis gewesen?

Concepts · fünf Kernbotschaften

1. LLMs erzeugen plausible Outputs, keine garantierten Wahrheiten.
2. Kontext bestimmt wesentlich die Qualität des Ergebnisses.
3. Context Engineering ist wichtiger als magische Prompt-Formeln.
4. Nicht jede Aufgabe gehört ins LLM.
5. Gutes AI Engineering reduziert den Bereich, in dem wir dem Modell vertrauen müssen.

6 · Concrete Practice: 1–1–1

Timing: ca. 5 Minuten

Jeder schreibt für sich auf:

- **1 Erkenntnis**, die hängen geblieben ist
- **1 Sache**, die er beim nächsten Einsatz von AI anders machen möchte
- **1 offene Frage**, die noch besteht

Danach kurzer Austausch im Pair.

Optional

Offene Fragen einsammeln und als Input für die nächsten Module verwenden.

Übergang zum nächsten Modul

Heute haben wir uns angesehen, wie diese Systeme funktionieren und welche Grenzen sie haben. Im nächsten Modul drehen wir die Perspektive um: Wie nutzen wir diese Eigenschaften ganz konkret beim Entwickeln, ohne die Kontrolle über unseren Code abzugeben?

Nächstes Modul

AI-Assisted Coding: Staying in Control

Understand → Code → Verify → Improve → Scale → Build